

Your peer sets your step time: spatial GPU partitioning is pair-specific, not width-specific

Draft

Claims traced to docs/claims-and-evidence.md

Abstract

We partition a single AMD GPU between concurrent diffusion tenants using CU masks and a step-granular scheduler. Any such scheduler needs to know what a step costs at q units and how much more it costs when a peer holds the rest. We report that the second quantity is not a function of the split at all.

It is set by which peer. On gfx1201, SDXL beside CogVideoX-2b pays 1.689 $\times$ its solo step at 4+28 and 26.137 $\times$ at 28+4 — the penalty rises as the image tenant’s own slice *widens*, because its co-run step obeys $corun_{sdxl} = corun_{cog} + 0.54 solo_{sdxl}$, with $k \in [0.531, 0.560]$ over six runs. The dominant term is the video tenant’s step, which the image tenant’s width does not change. Fitted on three splits and tested out of sample on the two not yet run, predicting 1736 and 3369 ms against 1659.7 and 3213.2 measured. Going from 4 units to 28 improves the image tenant’s solo step 4.1 $\times$ and its co-run step 3.8 $\times$ worse. It is one-directional: the video tenant pays 1.006–1.064 beside SDXL and 1.276 beside another CogVideoX. Against rotation — the whole die to each tenant half the time — the pair’s aggregate step rate is 5.608/s against 0.629–2.889/s partitioned, so the best split is 1.94 $\times$ worse and the worst 8.9 $\times$.

It counts peers, not busy die. A slice pays 1.050, 1.367, 2.318, 4.520 at 1, 2, 4 and 8 equal ways on gfx1201 and 1.024, 1.247, 1.507, 2.245 on gfx90a. Four ways is +77.3% over the pairwise entry for a slice of the same width.

It stops at the process boundary. Two processes on disjoint 16+16 masks pay 1.97 $\times$ each; giving both the whole die instead costs 2.10 $\times$. Two *threads* of one process on those same halves pay 1.37 $\times$. In aggregate step rate, masking across processes costs 24% against not masking, so the mechanism is available only to a run-time holding every tenant in one address space.

And for same-model tenants it is drawn, not computed. The penalty at 16+16 is bistable: either 1.297 $\times$ or 1.949 $\times$, latched per process, identifiable from a single step, slow in 16.9%, [11.4%, 23.6%] of 160 processes. It is absent on gfx90a.

The scheduling consequence is negative and pre-registered. On the mismatched workload our grid was designed around — 405 cells, nine policies — the method is 0.56% worse than the strongest baseline against a 20% target. Strict priority, which never partitions at all, beats the adaptive scheduler on urgent miss rate by +134% under arrivals and +84.5% under backlog. A dual-ledger probe that appeared to cut urgent miss by 13% was avoiding a conservative envelope we had added ourselves; with the envelope removed its benefit is +0.41%, [-0.0211, +0.0256].

Finally, a methodological result we would report even if the rest had gone the other way. Three separate cost models here were beautiful, quantitative and wrong, each an artifact of a deferred-event instrument that repeats one stale reading when the CPU runs ahead — including one fitting five splits to within 3–9% over a 3 $\times$ range. The control that catches all three is cheap and was missing

from every one: re-measure the solo *after* the co-run, with the peers idle. A penalty that does not vanish when the peers stop is not a penalty.

1 Introduction

Serving a latency-sensitive image request beside a long-running video generation on one GPU is a scheduling problem with an unusual property: the two tenants can be given *disjoint sets of compute units* rather than alternating slices of time. On AMD hardware this is `hipExtStreamCreateWithCUMask`; on NVIDIA it is the SM mask. Spatial partitioning is attractive because neither tenant has to be suspended, and a suspended diffusion step is not free to resume.

The premise of any such scheduler is a cost model: how much does a step cost at q units, and how much more does it cost when a peer holds the other $32 - q$? This paper is about the second question, and our answer is that it is not a question about q . The externality is a property of the *pair* — which models, how many of them, and whether they share an address space — and for a mismatched pair it is dominated by a term the tenant’s own width cannot move. A scheduler that models it as $f(q, 32 - q)$, as ours did and as a cost-model-driven partitioner must, is not approximating this hardware; it is describing a different one.

Contributions.

- (1) A measurement of the co-run penalty on gfx1201 showing it is bistable and latched per process — drawn rather than computed from the split (§3).
- (2) The paper’s central result: for a mismatched pair the co-run step time is set by the *slower* tenant, so widening the faster tenant’s slice makes it worse, and every split loses to rotation. Five splits on gfx1201, pre-registered, with the law tested out of sample (§4).
- (3) Two further ways the penalty escapes $f(q, 32 - q)$: it counts peers rather than busy die, at 1, 2, 4 and 8 ways on two architectures; and it does not cross the process boundary, so the mechanism exists only inside one address space (§4).
- (4) Three negatives. Two are about our own design: a probe whose measured benefit is entirely the avoidance of a conservative envelope we added (§6), and a pre-registered failure on the workload the grid was built around (§7). The third is that strict priority — no partitioning at all — beats the adaptive scheduler on this workload.
- (5) A methodological account of three cost models that were beautiful, quantitative and wrong for the same instrument reason, and of the one control that catches all three (§8).

What this paper does not claim. That spatial partitioning is useless. Masking is exact here, byte-for-byte reproducible across re-grants, and beats two unmasked full-die streams. What we claim is that its cost cannot be predicted from widths, that on a mismatched

state	processes	mean	range
fast	23 of 30	1.297	1.287–1.303
slow	7 of 30	1.949	1.923–1.962

Table 1: Co-run cost as a multiple of the solo step, one campaign. The clusters do not overlap.

pair it loses to time-slicing, and that a scheduler must therefore keep temporal isolation as a first-class action and refuse configurations it has no measurement for.

2 Setup

One Radeon AI PRO R9700 (gfx1201, 32 maskable units, 34.2 GB). SDXL at 768×768 ; CogVideoX-2b at 480×720 , 9 frames. A step-granular executor exposes prepare, run_step, suspend, resume and finalize; suspension captures the latent, the step index, the scheduler’s internal cursor and the generator state. Interrupting a request between every step and re-granting it at a different mask width produces a latent identical, byte for byte, to the uninterrupted baseline’s.

Masks are installed per stream and read back every time. A stream that quietly carried the whole die would produce an unusually *low* externality, which reads as good news, so the readback is evidence rather than a sanity check.

Measuring a step. Step cost is device time between two CUDA events bracketing the step on its own stream. Reading those events with a per-step synchronize drains the pipeline and charges the drain to the measurement: it made a 0.1155 s step read as 0.256 s. Reading them one step late avoids the drain but introduces a failure discussed in §8.

3 The co-run penalty is bistable

Two SDXL tenants on disjoint 16+16 masks pay one of two costs.

Three properties make this actionable rather than merely annoying.

It latches. No process was observed in both states: 0 of 8 processes changed across nine later episodes each, although every episode built fresh adapters and re-acquired its streams. Re-forming a pairing does not redraw.

It is per mask pair. Interposing an 8+24 pairing never disturbed the 16+16 state (16 of 16 checks), and 8+24 carried its own independent draw.

One step identifies it. Comparing each episode’s first step against its own median gives a mean absolute error of 0.00% over 72 episodes; the per-step series inside an episode is constant.

Pooled over 160 processes the slow state occurs in 27, 16.9%, Clopper–Pearson [11.4%, 23.6%]. The rate is *not* stable across campaigns — 22.7%, 35.0%, 8.3%, 11.7%, with Fisher $p = 0.025$ between the extremes — and we report no predictor. Five were proposed and retracted: working set, power cap, uncontrollable bistability, launch stagger, and prior use of a full-die mask.

It is RDNA4’s, not AMD’s. On an MI250X (gfx90a, CDNA2, 104 units) the same measurement — same instrument, same model, the same half-and-half split at 52+52, 32 processes rotating across

split (sdxl+cog)	SDXL solo	SDXL co-run	SDXL ext	same model	Cog ext
4+28	506.2	855.1	1.689	1.139	1.064
8+24	262.1	780.0	2.975	1.235	1.057
16+16	153.7	920.1	5.987	1.324	1.050
24+8	124.0	1659.7	13.386	1.202	1.025
28+4	122.9	3213.2	26.137	1.115	1.006

Table 2: Milliseconds per denoising step, wall clock with a per-adapter drain, last of six episodes. *same model* is SDXL beside SDXL at the identical width, measured in the same session on the same instrument. Every control returns to within 3.2% of the pre-episode solo and every solo is within 3% of the measured quota curve.

four GCDs — gives 1.2336 with a standard deviation of 0.0030 and nothing above the slow threshold. At the gfx1201 rate that has probability 0.0027. The four GCDs do differ from one another, deterministically, by 0.6% with non-overlapping ranges, so the instrument resolves far less than a 50% bimodality on that machine. Partitioning still pays there, +5.6% against rotation. The mechanism generalises; the pathology does not.

The consequence for a cost model is direct. Against rotation at 2×108.9 ms, the fast state makes partitioning worth +11.5% and the slow state worth −25.5%. A model with one externality per width pair is calibrating against a draw it cannot see.

4 The slower tenant sets the pace

Table 2 is the paper’s central measurement. Read down the SDXL column: the image tenant’s slice grows from 4 units to 28, its solo step improves 4.1×, and its co-run step gets 3.8× worse. Against its own same-model control at the identical width it pays 1.48× at 4+28 and 23.43× at 28+4.

The reason is visible in the two co-run columns. Across all six mismatched runs, including one with the offsets reversed,

$$corun_{sdxl} = corun_{cog} + k \text{ solo}_{sdxl}, \quad k \in [0.531, 0.560], \quad (1)$$

mean 0.543, spread 5.5% of it. The dominant term is the video tenant’s step. The image tenant’s width does not appear in it, and widening the image tenant’s slice enters only by narrowing the video tenant’s, which makes the dominant term larger: $corun_{cog}$ runs from 581 ms at 28 units to 3148 ms at 4. More die, for this tenant, on this pair, is strictly worse.

Equation 1 was fitted on the first three splits to land and written down, with its predictions for the two not yet run, before those ran: 1736 and 3369 ms against 1659.7 and 3213.2 measured, 4.4% and 4.6% out. We record this because a five-point fit to five points is not evidence of a law, and because this project has published a beautiful five-split fit before that was fitting an instrument (§8).

It is one-directional, and it follows the model. CogVideoX pays 1.006–1.064 beside SDXL and 1.276 beside another CogVideoX at the same width, so the mismatch is 0.82× cheaper for it. Nothing here is symmetric contention: the short-step tenant pays the long-step tenant’s step. Reversing the offsets at 16+16, so the models

exchange halves of the die, moves the penalty by 0.06 — it travels with the tenant, not with the position.

Against rotation. A side beats rotation — the whole die for half the wall clock — iff its co-run step is under twice its whole-die solo. The image tenant loses at all five splits. The video tenant wins at 4+28, 8+24 and 16+16 and loses at the two where its own slice is narrow. For the pair together, aggregate step rate is 5.608/s under rotation against 2.889, 2.859, 2.279, 1.230 and 0.629/s partitioned: the best split 1.94× worse, the worst 8.9×.

Blocked or slowed, we do not say. The image tenant’s event-timed step tracks its own solo at every split while its wall clock reads up to 26× that; the video tenant’s two instruments agree to 0.1%. Either the fast tenant’s deferred event reading is stale — the failure of §8, and the fast tenant is exactly the one whose CPU races — or its kernels are unslowed and its thread is blocked between steps. The throughput result is the same either way, 13 steps in 12.0 s against 2.0 s alone, but the mechanism is not, and our stored series cannot separate them.

5 The penalty counts peers, and stops at the process boundary

Two more measurements say the externality is not $f(q, 32 - q)$, and neither needs a mismatched pair.

Peers, not busy die. With N equal slices of one model covering the whole die, a slice pays 1.050, 1.367, 2.318 and 4.520 at $N = 1, 2, 4, 8$ on gfx1201, and 1.024, 1.247, 1.507 and 2.245 on gfx90a. In every case the die is fully subscribed and only the number of peers changes. Four slices of 8 units pay 77.3% more than the pairwise table’s entry for an 8-unit slice with one 24-unit peer. The $N = 2$ cell is the control that licenses the rest: there the arrangement is the pairwise one, and it agrees with the independently measured pairwise entry to 2.4%.

Not across processes. Every co-run number above, and every one we know of in this line of work, is measured with threads of one process. Deployed multi-tenant serving is normally one server per tenant. Two processes on disjoint 16+16 masks pay 1.97× each; giving both processes the whole die instead costs 2.10× — close enough to 2× to be the hardware scheduler time-slicing them. The mask still applies, verified against the quota curve at 144.6 versus 112.8 ms unmasked, so it caps each process at half the units and the time-slicing happens anyway. In aggregate step rate: one process alone on the whole die 8.87/s, two threads of one process at 16+16 9.54/s, two processes with the whole die each 8.45/s, two processes on disjoint halves 6.46/s. Masking across processes costs 24% and buys nothing.

6 A scheduler that measures, and what it was really doing

The runtime charges each tenant from the *measured* cost of the step it just ran, never from the cost model. Charging from prediction would make the accounting exact by construction and blind. The policy pairs tenants by matched step cost and probes: if a formed pairing’s observed step exceeds its paired expectation by a tolerance, the pairing is dropped for a round. The runtime adds a safety

policy	envelope on	envelope off
static SM partition	0.3011	0.2415
EDF	0.2999	0.2574
step-matched pairing	0.3011	0.2415
probe	0.2619	0.2353

Table 3: Urgent SLO miss rate, 9 paired seeds. Every policy improves with the envelope off; all four intervals exclude zero.

envelope that falls back to a serial action when a pairing has no measured profile or when drift exceeds 15%.

Decomposed with three arms from one code path — pairing only, plus the deadline actions, plus the probe — paired by seed, the probe carried −13.14% in the fast state and −14.20% in the slow one, with the deadline actions null in both, and it cut serial fallbacks by 73–81%.

Then we removed the envelope. Every policy improved. And the probe’s advantage over step-matched pairing went from −13.01%, [−0.0668, −0.0132], to +0.41%, [−0.0211, +0.0256] — an interval spanning zero with the point estimate on the wrong side.

So the mechanism was identified correctly and its significance was backwards. The probe does cut serial fallbacks; those fallbacks are harmful; and they are harmful because they are ours. The envelope fired on drift measured as $|\text{observed/expected} - 1|$, which treats a pairing arriving *cheaper* than predicted as a reason to take the whole die serial. Correcting it to fire on over-run only removed half the firings, and removing it entirely was better still. The envelope’s other branch, a fallback when the cost model has no entry for a pairing, never fired in any workload we ran, so it is untested rather than refuted.

The best configuration we measured is the envelope off and no probe: step-matched pairing at 0.2276 against 0.2286 with the probe.

Robustness. Injecting predictor error into the belief the policy reads, and nowhere else, gives zero safety failures at $\pm 20\%$ across 134 cells with the envelope both on and off, where a safety failure is defined in code as over-committing the die, charging from the cost model when a measurement existed, or charging a measurement taken at another quota.

7 A pre-registered negative

On the mismatched workload — 405 cells, nine policies, five seeds, loads {0.6, 0.85, 1.05} and bursts {2, 4, 8} — the method is 0.56% worse than the strongest non-oracle baseline, [+0.0000, +0.0056] over 45 paired configurations, against a target of a 20% improvement.

It follows from §4, though not in the way we first wrote it down. We originally explained it by saying mismatched pairings cost 1.02–1.06, so a contention manager has nothing to manage. That explanation was built on a measurement we have since withdrawn. The replacement runs the other way: partitioning a mismatched pair is not cheap but ruinous for the faster tenant, so a policy whose action is to partition is choosing the worse arm, and a policy that measures more carefully still chooses it. The pairing itself does win

against the baselines that were in the grid — FCFS by 6.5%, static partitioning and EDF by about 3%, and at overload FCFS by 17% — which is why the baseline set mattered more than the policy set, as the next paragraph but one records.

The baseline that was missing. For most of this project’s life the “no partitioning” arm was whole-die time-slicing between tenants. Strict priority — the whole die to the deadline-carrying tenant whenever it has work, no partitioning ever, the obvious production policy — had never been run. It beats the adaptive scheduler on urgent miss rate by +134% under arrivals and +84.5% under backlog, intervals excluding zero under a seed cluster bootstrap, better in 15 of 20 cells and tied in 3; under arrivals it dominates, with identical video goodput and identical Jain fairness. We then tested the two defects that could have explained it — a round barrier and a two-action space — in a 240-cell factorial, and neither does: every cell loses to priority by +83% to +434%, and six actions are far worse than two.

How much of the grid could separate anything. “45 paired configurations” overstates the evidence, so we say what it is. The probe and step-matched pairing produce *identical* miss rates in 42 of the 45; three differ, all in the same direction. A bootstrap resample that draws none of the three has probability $(42/45)^{45} = 4.5\%$, above 2.5%, so the interval’s lower bound is zero as arithmetic and cannot be read as significance — the grid shows the probe does not help, not that it hurts. The comparisons above are wider: pairing differs from FCFS in 21 configurations, from static partitioning in 16, from EDF in 17. Two tenants are both runnable for only 26.1% of the horizon at load 0.6 and 44.9% at 1.05, and three of the 45 configurations draw no video tenant at all, so most of this grid could not have separated any two partitioning policies whatever they did.

We state this as a negative because it was registered as one. The comparison target was fixed by rule — best baseline mean, computed not chosen — because the same method beats FCFS by 30%, the strongest baseline by 22%, and its own ablation by 4.5% on a single cell, and which opponent a headline uses is the easiest thing to lose between a table and a sentence.

8 Three instruments that fitted artifacts

Reading a step’s events one step late avoids draining the pipeline, but the read only lands once the previous step’s events have retired. When the measured side’s steps are short it never lands, and the variable holds one stale value for an entire episode. The side whose steps are short is the side whose CPU runs ahead, which in a mismatched pair is exactly the side whose penalty is being measured.

The same defect produced three published cost models, each quantitative and each wrong.

One. A two-episode “serialisation transient” in which each side appeared to cost the sum of the two solo steps, fitting five splits to within 3–9% over a $3\times$ range of that sum. A real fit to a stale variable.

Two. A wall clock introduced to replace the stale events, but stopped without synchronising the device, so it measured the CPU’s submission rate: 14 steps of a 269 ms quota read 156 ms. The N-way

penalty of \$5 was withdrawn on that reading and restored the next day when the clock was drained before it was stopped.

Three. The claim this paper’s \$4 replaces: mismatched tenants costing 1.00–1.06 per side and Pareto-dominating rotation at every split. Its own published text contains the refutation, read at the time as a warm-up transient — a first co-run episode of 988 ms per step, “exactly its own 183 ms solo step plus one whole 805 ms CogVideoX step, with no spread at all”. No spread at all is the signature of a variable that is not being updated. What we now measure at that split is 5.987, and equation 1 is the 988 ms account with a coefficient of 0.543 instead of 1 and as the steady state rather than a transient.

The control that catches all three is cheap and was missing from every one of them: after the co-run episodes, re-measure the solo with the peers idle, then again after dropping the allocator cache, then again with the peer streams destroyed. A penalty that does not vanish when the peers stop is not a penalty. Applying it retroactively disqualifies the comparator set we had intended to use for \$4 — at eight ways its post-episode solo reads 2048 ms against a pre-episode 504 — which is why that section measures its own same-model baseline in the same session.

What caught it was not reasoning. Running a second instrument — events bracketing the same step on the same stream, which physically *contain* the first — over the same steps in the same process gave 1.022 where the first gave 10.888. Containment made the contradiction unarguable.

We record the working rules this produced, because the same class of error — a property of the measurement presenting itself as a property of the thing measured — occurred nine times: measure the null before varying parameters; state the discriminating criterion and its direction before running; a single co-run measurement is not evidence; the measurement architecture must match the claimed deployment architecture; and make the nuisance variable identical across arms rather than trusting it to balance — seed with position, deadline with arm, and co-run state with arm each confounded a campaign before that was routine.

9 Threats to validity

Two SKUs, one vendor. Results are from gfx1201 (RDNA4, 32 maskable units) and gfx90a (CDNA2, 104 per GCD). The two disagree in ways that matter: bistability is present on the first and absent on the second, and the aggregate solo throughput of N slices peaks at four ways on gfx90a where it is monotone on gfx1201, so the Amdahl form our cost model assumes is refuted there outright. Nothing rules out that the whole family of results depends on AMD’s queueing and arbitration. No number of additional cells on the same two cards substitutes for cross-vendor agreement.

Two models, one workpoint each. SDXL at 768×768 and CogVideoX-2b at $480 \times 720 \times 9$. Equation 1’s coefficient is fitted across five splits of one pair; whether k is a constant of the hardware, of the step-length ratio, or of these two models is not established. The obvious next measurement is a third model with an intermediate step length.

One process. §5 shows the mechanism does not survive the process boundary, so every quantitative result here describes a run-time that holds all tenants in one address space. For a deployment of one server per tenant, the finding is the 24% loss, not the table.

Same-model co-location is not the primary workload. The probe results of §6 are measured there because the mechanism is inert on the mismatched one.

Small n on the slow state. Ten processes. The interval excludes zero; it is not tight.

An unexplained ablation. Making the runtime blind to the externality term *reduced* serial fallbacks where the mechanism and a unit test both say it should increase them. Under investigation.

10 Conclusion

The claim this work set out to make — that spatial partitioning raises utilisation by a fixed factor — is false as stated, and so was our first replacement for it. The factor is not fixed, and it is not a function of the split. It is drawn per process for same-model tenants, it grows with the number of peers rather than with how busy the die is, it vanishes across the process boundary, and for a mismatched

pair it is set by the slower tenant’s step, so the faster tenant gets worse as you give it more of the die.

That last one is why our scheduler lost. A partitioner needs *externality*($q, 32 - q$); this hardware supplies something whose arguments include which models, how many, and in whose address space. Our policies asked the first question, and every negative in this paper — the pre-registered grid, the probe, the loss to strict priority — is what that costs. A scheduler that wants to use spatial partitioning here needs pair-specific measurements, an explicit refusal for configurations it has none for, and time-slicing kept as a first-class action rather than as the baseline it is assumed to beat.

We report our own retractions in full because three of them came from one instrument defect and one control would have caught all three, and because the version of this paper that would have been submitted a month ago had a correct-looking quantitative account of a phenomenon that was not happening.